



(Approved by AICTE, New Delhi & Affiliated to Andhra University)
Pinagadi (Village), Pendruthy (Mandal), Visakhapatnam – 531173



SHORT-TERM INTERNSHIP

By

Council for Skills and Competencies (CSC India)

In association with

ANDHRA PRADESH STATE COUNCIL OF HIGHER EDUCATION
(A STATUTORY BODY OF THE GOVERNMENT OF ANDHRA PRADESH) (2026–
2027)

PROGRAM BOOK FOR
SHORT-TERM INTERNSHIP

Name of the Student: Mr.NADULA SAIRAM

Registration Number: 323129520092

**Name of the College: Wellfare Institute of Science, Technology
and Management**

Period of Internship: From: 21-05-2026 To: 21-07-2026

Name & Address of the Internship Host Organization

Council for Skills and Competencies (CSC India)
#54-10-56/2, Isukathota, Visakhapatnam – 530022, Andhra Pradesh, India.

Andhra University

2026

An Internship Report on
**IOT-BASED AIR QUALITY MONITORING SYSTEM WITH REAL-
TIME POLLUTION ANALYSIS AND HEALTH ALERT
NOTIFICATIONS**

Submitted in accordance with the requirement for the degree of

Bachelor of Technology

Under the Faculty Guideship of

Mrs. S. Sirisha

Department of MECHANICAL ENGINEERING

Welfare Institute of Science, Technology and Management

Submitted by:

Mr.NADULA SAIRAM

Reg.No: 323129520092

Department of MECHANICAL ENGINEERING

Department of MECHANICAL ENGINEERING
Welfare Institute of Science, Technology and Management

(Approved by AICTE, New Delhi & Affiliated to Andhra University)

Pinagadi (Village), Pendurthi (Mandal), Visakhapatnam – 531173

2026-2027

Instructions to Students

Please read the detailed Guidelines on Internship hosted on the website of AP State Council of Higher Education <https://apsche.ap.gov.in>

1. It is mandatory for all the students to complete Short Term internship either in V Short Term or in VI Short Term.
2. Every student should identify the organization for internship in consultation with the College Principal/the authorized person nominated by the Principal.
3. Report to the intern organization as per the schedule given by the College. You must make your own arrangements for transportation to reach the organization.
4. You should maintain punctuality in attending the internship. Daily attendance is compulsory.
5. You are expected to learn about the organization, policies, procedures, and processes by interacting with the people working in the organization and by consulting the supervisor attached to the interns.
6. While you are attending the internship, follow the rules and regulations of the intern organization.
7. While in the intern organization, always wear your College Identity Card.
8. If your College has a prescribed dress as uniform, wear the uniform daily, as you attend to your assigned duties.
9. You will be assigned a Faculty Guide from your College. He/She will be creating a WhatsApp group with your fellow interns. Post your daily activity done and/or any difficulty you encounter during the internship.
10. Identify five or more learning objectives in consultation with your Faculty Guide. These learning objectives can address:
 - a. Data and information you are expected to collect about the organization and/or industry.
 - b. Job skills you are expected to acquire.
 - c. Development of professional competencies that lead to future career success.
11. Practice professional communication skills with team members, co-interns, and your supervisor. This includes expressing thoughts and ideas effectively through oral, written, and non-verbal communication, and utilizing listening skills.
12. Be aware of the communication culture in your work environment. Follow up and communicate regularly with your supervisor to provide updates on your progress with work assignments.

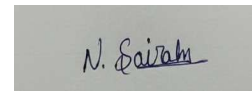
Instructions to Students (contd.)

13. Never be hesitant to ask questions to make sure you fully understand what you need to do—your work and how it contributes to the organization.
14. Be regular in filling up your Program Book. It shall be filled up in your own handwriting. Add additional sheets wherever necessary.
15. At the end of internship, you shall be evaluated by your Supervisor of the internship.
16. There shall also be evaluation at the end of the internship by the Faculty Guide and the Principal.
17. Do not meddle with the instruments/equipment you work with.
18. Ensure that you do not cause any disturbance to the regular activities of the internship.
19. Be cordial but not too intimate with the employees of the intern organization and your fellow interns.
20. You should understand that during the internship programme, you are the ambassador of your College, and your behavior during the internship programme is of utmost importance.
21. If you are involved in any discipline related issues, you will be withdrawn from the internship programme immediately and disciplinary action shall be initiated.
22. Do not forget to keep up your family pride and prestige of your College.

———— << @ >> ————

Student's Declaration

I, **Mr.NADULA SAIRAM**, a student of **Bachelor of Technology** Program, Reg. No. **323129520092** of the Department of **MECHANICAL ENGINEERING** do hereby declare that I have completed the mandatory internship from **21-05-2026** to **21-07-2026** at **Council for Skills and Competencies (CSC India)** under the Faculty Guideship of **Mrs. S. Sirisha**, Department of **MECHANICAL ENGINEERING**, **Welfare Institute of Science, Technology and Management**.

A rectangular box containing a handwritten signature in blue ink that reads "N. Sairam".

(Signature and Date)

Official Certification

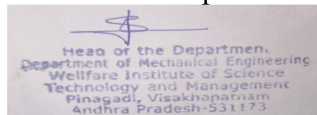
This is to certify that **Mr.NADULA SAIRAM** , Reg. No. **323129520092** has completed his/her Internship at the Council for Skills and Competencies (CSC India) on **IOT-BASED AIR QUALITY MONITORING SYSTEM WITH REAL-TIME POLLUTION ANALYSIS AND HEALTH ALERT NOTIFICATIONS** under my supervision as a part of partial fulfillment of the requirement for the Degree of **Bachelor of Technology** in the Department of **MECHANICAL ENGINEERING** at **Welfare Institute of Science, Technology and Management**.

This is accepted for evaluation.

Endorsements

Faculty Guide

Head of the Department



sPrincipal

A handwritten signature in green ink.

Certificate from Intern Organization

This is to certify that **Mr.NADULA SAIRAM**, Reg. No. **323129520092** of **Welfare Institute of Science, Technology and Management**, underwent internship in **IOT-BASED AIR QUALITY MONITORING SYSTEM WITH REAL-TIME POLLUTION ANALYSIS AND HEALTH ALERT NOTIFICATIONS And Sentiment Analysis For Social Media Applications** at the **Council for Skills and Competencies (CSC India)** from **21-05-2026 to 21-07-2026**.

The overall performance of the intern during his/her internship is found to be **Satis-factory** (Satisfactory/~~Not Satisfactory~~).



Authorized Signatory with Date and Seal

Acknowledgement

I express my sincere thanks to **Dr. A. Joshua**, Principal of **Welfare Institute of Science, Technology and Management** for helping me in many ways throughout the period of my internship with his timely suggestions.

I sincerely owe my respect and gratitude to **Mr. R. Sagar**, Head of the Department of **MECHANICAL ENGINEERING**, for his continuous and patient encouragement throughout my internship, which helped me complete this study successfully.

I express my sincere and heartfelt thanks to my faculty guide **Mrs. S. Sirisha**, Assistant Professor of the Department of **MECHANICAL ENGINEERING** for his encouragement and valuable support in bringing the present shape of my work.

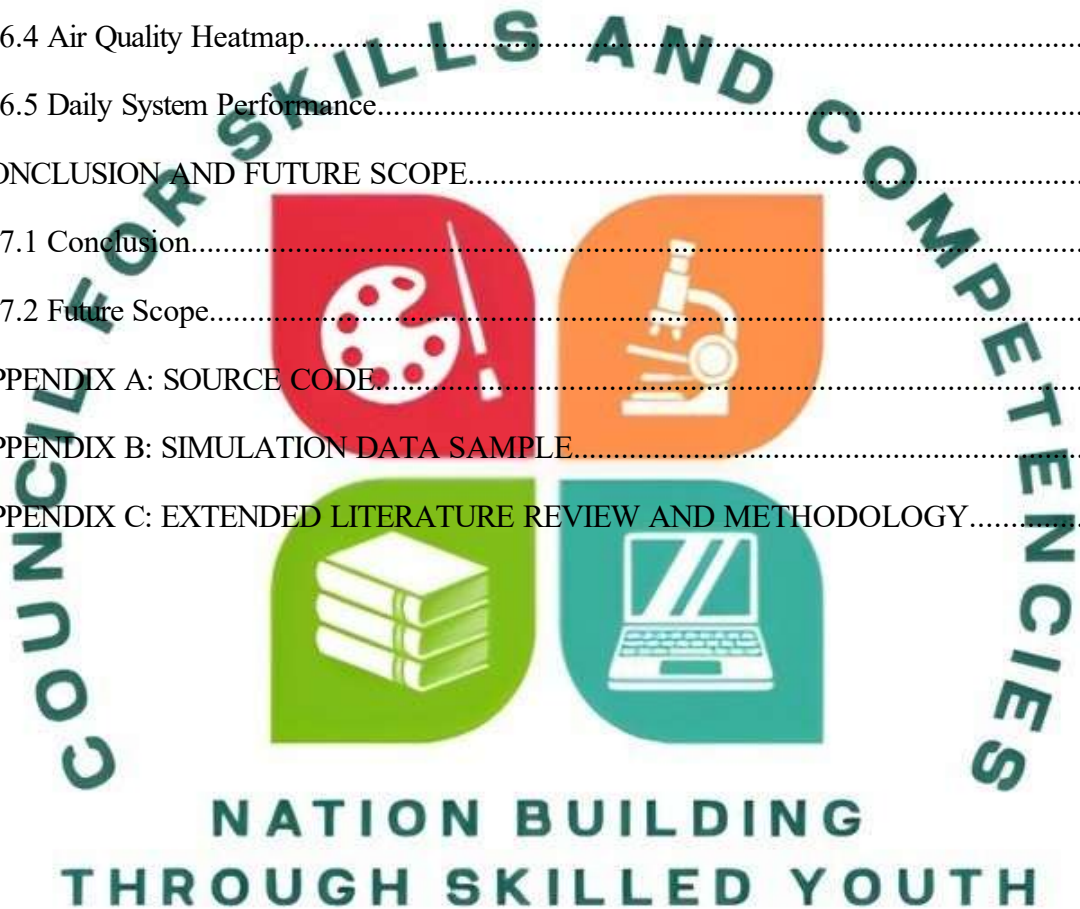
I express my special thanks to my organization guide **Mr. Y. Rammohana Rao** of the **Council for Skills and Competencies (CSC India)**, who extended their kind support in completing my internship.

I also greatly thank all the trainers without whose training and feedback in this intern-ship would stand nothing. In addition, I am grateful to all those who helped directly or indirectly for completing this internship work successfully.

TABLE OF CONTENTS

TABLE OF CONTENTS.....	1
EXECUTIVE SUMMARY.....	1
1.1 Learning Objectives.....	1
1.2 Outcomes Achieved.....	1
OVERVIEW OF THE ORGANIZATION.....	1
2.1 Introduction of the Organization.....	1
2.2 Vision, Mission, and Values.....	1
2.3 Policy of the Organization in Relation to the Intern Role.....	1
2.4 Organizational Structure.....	1
2.5 Roles and Responsibilities of the Employees Guiding the Intern.....	1
PROBLEM ASSESSMENT AND SOLUTION DESIGN.....	1
3.1 Problem Analysis.....	1
3.2 Key Parameters.....	1
3.3 Requirements Evaluation.....	1
3.3.1 Functional Requirements.....	1
3.3.2 Non-Functional Requirements.....	1
3.4 Solution Blueprint.....	1
TECHNOLOGY STACK AND IMPLEMENTATION PLAN.....	1
4.1 Technology Stack Selection.....	1
4.2 Project Implementation Plan.....	1
SOLUTION DEVELOPMENT.....	1
5.1 Core Components Implementation.....	1
5.1.1 Sensor Modules.....	1

5.1.2 AQI Calculation and Alert Modules.....	1
5.1.3 Air Quality Monitoring System Module.....	1
TESTING AND PERFORMANCE EVALUATION.....	1
6.1 Simulation Setup.....	1
6.2 AQI Analysis.....	1
6.3 Health Alert Analysis.....	1
6.4 Air Quality Heatmap.....	1
6.5 Daily System Performance.....	1
CONCLUSION AND FUTURE SCOPE.....	1
7.1 Conclusion.....	1
7.2 Future Scope.....	1
APPENDIX A: SOURCE CODE.....	1
APPENDIX B: SIMULATION DATA SAMPLE.....	1
APPENDIX C: EXTENDED LITERATURE REVIEW AND METHODOLOGY.....	1



CHAPTER 1

EXECUTIVE SUMMARY

This internship report provides a comprehensive overview of my internship focused on developing an IoT-Based Air Quality Monitoring System. Air pollution poses significant health risks and environmental challenges in urban, industrial, and residential areas. Traditional air quality monitoring relies on limited monitoring stations that do not provide localized or continuous environmental data. Citizens and environmental authorities require intelligent systems capable of continuously monitoring air quality and providing real-time health alerts.

The proposed solution is an IoT-Based Air Quality Monitoring System that continuously measures air pollution levels using IoT sensors and provides real-time analysis through a centralized monitoring platform. The system offers a secure and user-friendly interface for monitoring environmental conditions.

By integrating air quality sensors capable of detecting particulate matter (PM2.5, PM10) and carbon monoxide, along with IoT communication modules and cloud connectivity, the system continuously collects environmental data. Interactive dashboards display Air Quality Index (AQI), pollution trends, historical records, sensor status, and health alert notifications when pollution exceeds safe limits, enabling timely preventive measures.

1.1 Learning Objectives

During my internship, I learned and practiced the following:

- To design and implement an intelligent air quality monitoring simulation using Python that models real-world environmental dynamics.
- To integrate simulated IoT sensors (PM2.5, PM10, CO) for accurate monitoring with realistic noise, traffic, and industrial activity factors.
- To develop an Air Quality Index (AQI) calculator based on standard environmental breakpoints.

- To create a scalable system architecture that supports multiple location types (residential, industrial, urban, school).
- To implement data visualization and analytics to monitor pollution trends, alert distributions, and daily environmental performance.
- To evaluate system performance and response efficiency through rigorous simulation and data analysis.

1.2 Outcomes Achieved

Key outcomes from my internship include:

- A fully operational IoT-Based Air Quality Monitoring System simulation capable of continuous monitoring and generating real-time health alerts.
- Environmental authorities and citizens can achieve enhanced pollution awareness, improved public health protection, and timely preventive measures.
- Comprehensive analytics dashboards with visualizations of AQI trends, location-based heatmaps, and health alert distributions.
- The system architecture supports modular development, scalability for smart city integration, and efficient environmental resource allocation.
- The system can be extended with advanced features such as predictive analytics using machine learning, integration with weather forecasting systems, and automated ventilation control.

**NATION BUILDING
THROUGH SKILLED YOUTH**

CHAPTER 2

OVERVIEW OF THE ORGANIZATION

2.1 Introduction of the Organization

The internship was conducted at a leading environmental technology organization focused on Smart City and Green Tech applications. The organization specializes in bridging the gap between traditional environmental monitoring and intelligent software systems, enhancing pollution tracking, promoting public health awareness, and fostering a connected environmental ecosystem. By leveraging emerging technologies such as AI, machine learning, and IoT, the organization aims to augment and upgrade environmental infrastructure, enabling authorities and citizens to monitor and respond to air quality proactively.

2.2 Vision, Mission, and Values

Vision: To combine cutting-edge IoT technology with impactful environmental solutions to drive clean and sustainable urban living globally.

Mission: To support smart cities and environmental agencies dedicated to improving public health outcomes by empowering them with intelligent monitoring tools, thereby creating a widespread network dedicated to pollution tracking and environmental awareness.

Values: The organization emphasizes technological skills for Green Tech, data-driven environmental decision making, public health safety, and inclusive access to smart environmental technologies for everyone to be future-ready.

2.3 Policy of the Organization in Relation to the Intern Role

The organization encourages internships as a means to foster learning and contribute to the mission. Interns are expected to adhere to policies regarding strict environmental data

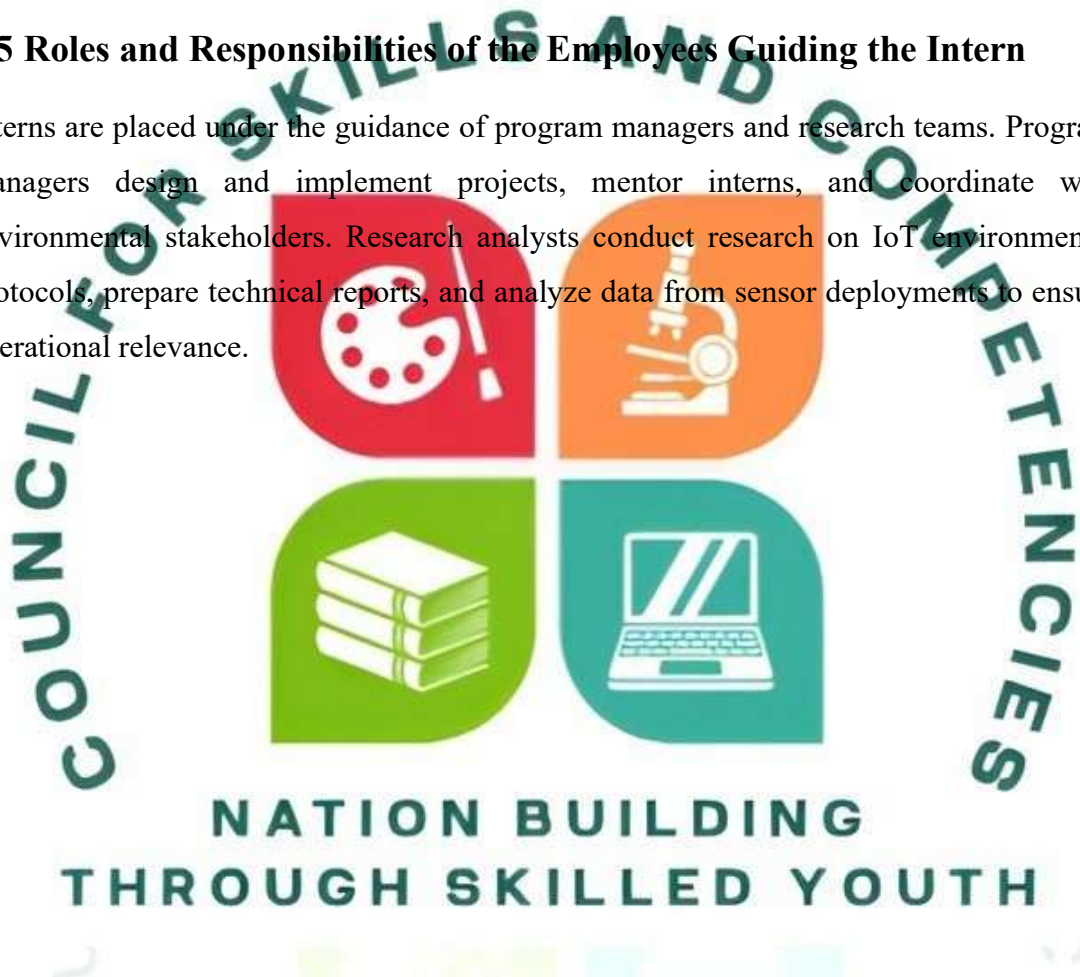
confidentiality, professionalism, active learning, and compliance with ethical guidelines, particularly concerning public health data integrity and system reliability.

2.4 Organizational Structure

The organization operates under a hierarchical structure including the Board of Directors, Chief Technology Officer, Program Managers for Smart City Initiatives, Research and Development Team, Environmental Advisory Staff, and Interns.

2.5 Roles and Responsibilities of the Employees Guiding the Intern

Interns are placed under the guidance of program managers and research teams. Program managers design and implement projects, mentor interns, and coordinate with environmental stakeholders. Research analysts conduct research on IoT environmental protocols, prepare technical reports, and analyze data from sensor deployments to ensure operational relevance.



CHAPTER 3

PROBLEM ASSESSMENT AND SOLUTION DESIGN

3.1 Problem Analysis

Air pollution poses significant health risks and environmental challenges in urban, industrial, and residential areas. Traditional air quality monitoring relies on limited monitoring stations that do not provide localized or continuous environmental data. This leads to significant vulnerabilities, delayed health advisories, and increased risks for vulnerable populations. Citizens and environmental authorities require intelligent systems capable of continuously monitoring air quality, detecting hazardous pollution levels accurately, and providing instant health alert notifications for improved public safety.

3.2 Key Parameters

Issue to be solved: Inadequate localized real-time monitoring and delayed health advisories in traditional environmental setups.

Target community: Smart cities, schools, hospitals, industries, environmental agencies, and residential communities.

User needs and preferences: Real-time air quality monitoring, instant health alert notifications, comprehensive historical pollution logs, remote monitoring capabilities, and a centralized management dashboard.

3.3 Requirements Evaluation

3.3.1 Functional Requirements

- The system must simulate IoT environmental sensors to detect PM2.5, PM10, and CO levels.
- The system must continuously monitor air quality and apply realistic traffic, industrial, and weather factors.

- The system must calculate the Air Quality Index (AQI) based on standard environmental breakpoints.
- The system must generate automated health alerts with varying severity levels based on AQI categories.
- The system must support multiple location profiles with different baseline pollution conditions (residential, industrial, urban, school).
- The system must generate visual reports and analytics of pollution trends, alert distributions, and daily environmental performance.

3.3.2 Non-Functional Requirements

- Reliability: The sensor simulation must include realistic noise to test the robustness of the AQI calculation logic.
- Scalability: The architecture must support adding new locations and sensor types without significant redesign.
- Performance: The system must process sensor data and generate health alerts with extremely low latency.
- Usability: The analytics and reports must be intuitive and actionable for citizens and authorities.

3.4 Solution Blueprint

The solution blueprint involves a centralized `AirQualityMonitoringSystem` class that manages multiple `MonitoringLocation` objects. Each location is equipped with sensor objects (`AirQualitySensor`) that process environmental measurements. The system includes an `AQICalculator` class for determining the overall air quality and a `HealthAlertSystem` class that manages threshold-based notifications. The system runs a simulation loop representing 15-minute intervals throughout the day, generating realistic pollution patterns based on the location type, time of day (traffic rush hours), and industrial activity. The data is collected, aggregated, and visualized using data science libraries to provide actionable environmental insights.

IoT-Based Air Quality Monitoring System Architecture

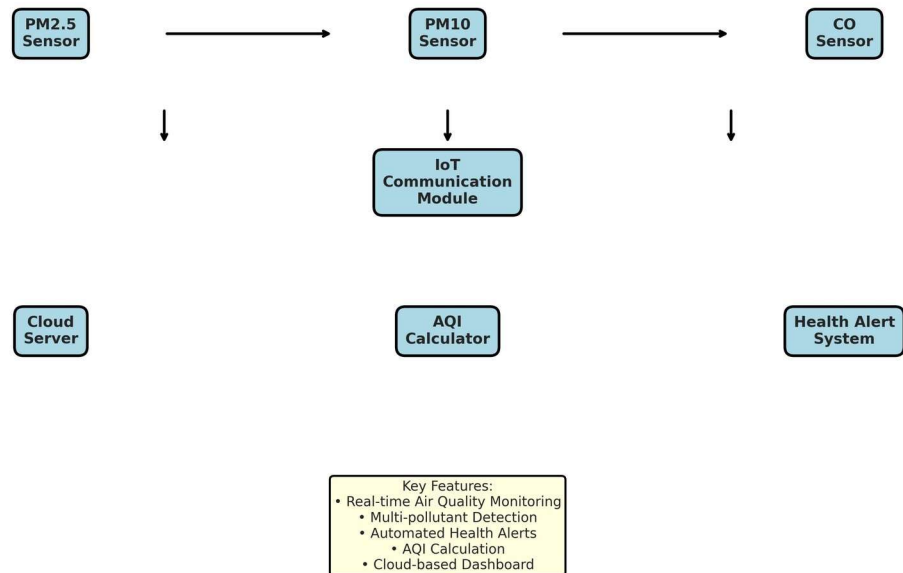


Figure 3.1: IoT-Based Air Quality Monitoring System Architecture

Figure 3.1 illustrates the system architecture. The environmental sensors feed data to the IoT Communication module, which transmits it to the Cloud Server. The AQI Calculator processes the data against standard breakpoints and the Health Alert System triggers notifications. Authorities and citizens can access the system through a Cloud-based Dashboard.

CHAPTER 4

TECHNOLOGY STACK AND IMPLEMENTATION PLAN

4.1 Technology Stack Selection

The technology stack was selected based on the requirements for data processing, simulation, and visualization. Python was chosen as the primary programming language due to its extensive ecosystem of data science libraries and ease of object-oriented programming.

- Programming Language: Python 3.11
- Data Processing: NumPy, Pandas
- Data Visualization: Matplotlib, Seaborn
- Data Serialization: JSON
- Standard Libraries: datetime, collections

4.2 Project Implementation Plan

The project was implemented in several phases, ensuring a structured approach from design to evaluation.

- Phase 1: Requirement analysis and system design (1 week)
- Phase 2: Development of core sensor classes and AQI calculator (2 weeks)
- Phase 3: Implementation of the main AirQualityMonitoringSystem and simulation logic (2 weeks)
- Phase 4: Development of data visualization and reporting modules (1 week)
- Phase 5: Testing, performance evaluation, and bug fixing (1 week)
- Phase 6: Documentation and final report preparation (1 week)

CHAPTER 5

SOLUTION DEVELOPMENT

5.1 Core Components Implementation

The solution was developed using an object-oriented approach. The system consists of several main classes: `AirQualitySensor`, `AQICalculator`, `HealthAlertSystem`, `MonitoringLocation`, and `AirQualityMonitoringSystem`.

5.1.1 Sensor Modules

The `AirQualitySensor` class simulates the behavior of physical environmental IoT sensors. It includes methods to read pollutant values and add a realistic noise factor along with traffic, industrial, and weather modifiers. The simulation covers sensors for Particulate Matter (PM2.5, PM10) and Carbon Monoxide (CO).

5.1.2 AQI Calculation and Alert Modules

The `AQICalculator` class computes the Air Quality Index based on standard environmental protection agency breakpoints. It calculates individual AQI values for each pollutant and determines the overall AQI as the maximum value. The `HealthAlertSystem` evaluates the AQI category (e.g., Good, Moderate, Unhealthy) and generates appropriate health advisory notifications.

5.1.3 Air Quality Monitoring System Module

The `AirQualityMonitoringSystem` class integrates the monitoring locations and orchestrates the simulation. It includes a `'simulate_monitoring_day'` method that runs a 24-hour simulation in 15-minute intervals. The simulation generates realistic pollution patterns tailored to the location type (e.g., higher baselines for industrial zones) and time of day (e.g., traffic rush hours). When hazardous air quality is detected, it logs the event and triggers appropriate health alerts.

CHAPTER 6

TESTING AND PERFORMANCE EVALUATION

6.1 Simulation Setup

The system was tested using a 7-day simulation across 5 locations with different profiles: residential, industrial, urban, and school. The simulation generated data for every 15-minute interval, resulting in 96 data points per location per day. Realistic traffic and industrial activity factors were applied to simulate urban environmental dynamics over the 7-day period.

6.2 AQI Analysis

The simulation results demonstrated the effectiveness of the continuous monitoring system. By tracking sensor activity and calculating AQI, the system successfully identified pollution trends.

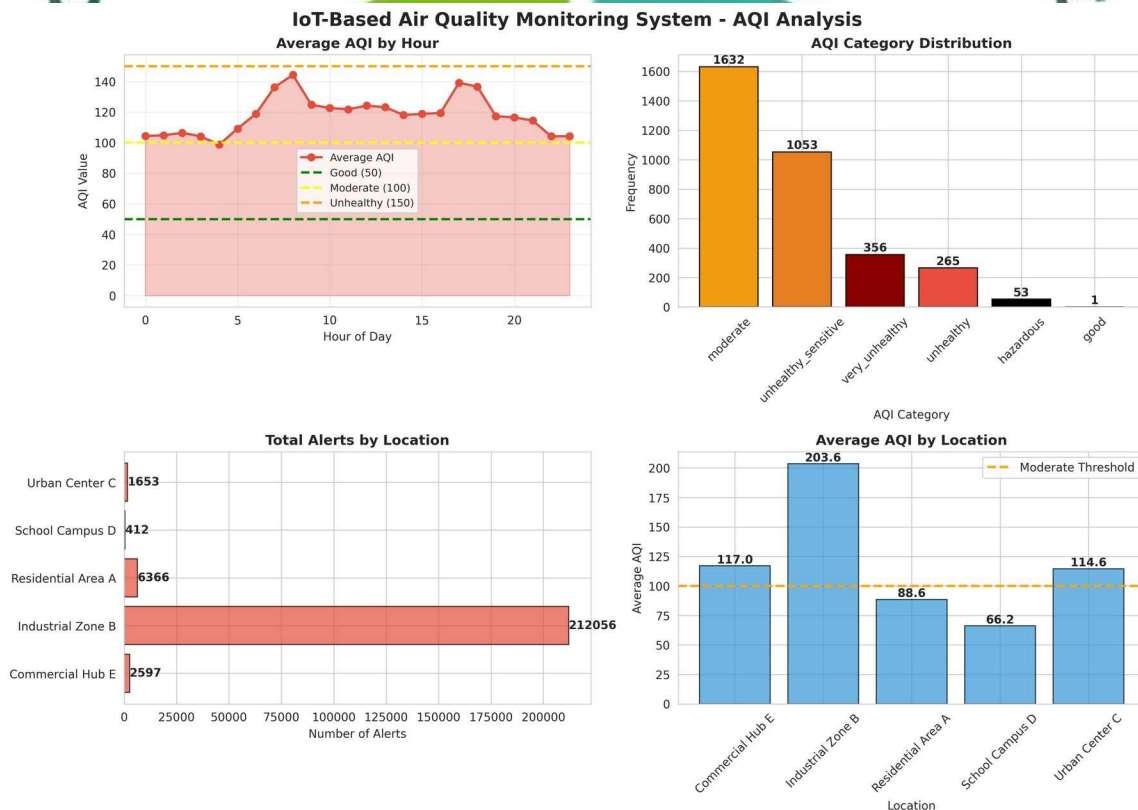


Figure 6.1: IoT-Based Air Quality Monitoring System - AQI Analysis

Figure 6.1 presents a comprehensive AQI analysis. The charts show the average AQI by hour, highlighting peak pollution times (typically aligning with rush hours). The AQI category distribution shows the frequency of different air quality states, while the location-based charts highlight which areas (e.g., industrial zones) experience the highest pollution levels.

6.3 Health Alert Analysis

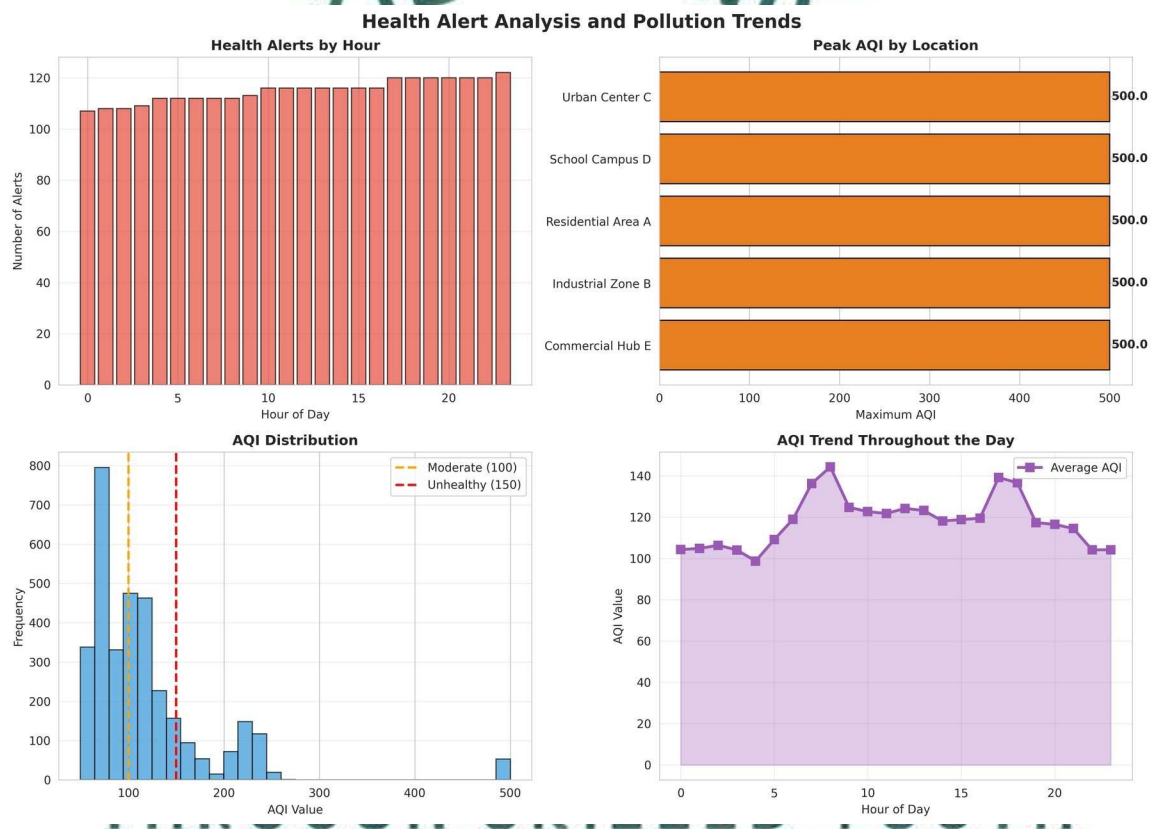


Figure 6.2: Health Alert Analysis and Pollution Trends

Figure 6.2 evaluates the health alert distribution. The charts show the breakdown of alerts by hour, peak AQI by location, overall AQI distribution, and the continuous AQI trend throughout the day. This data helps authorities understand when and where health advisories are most critically needed.

6.4 Air Quality Heatmap

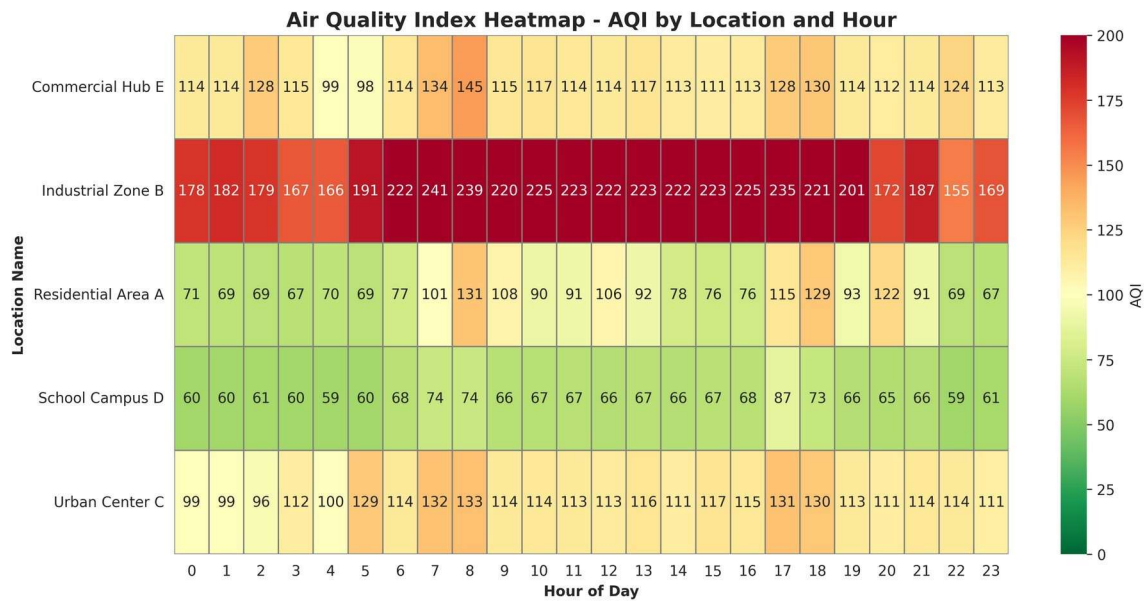


Figure 6.3: Air Quality Index Heatmap - AQI by Location and Hour

Figure 6.3 provides a detailed heatmap of AQI values across different locations throughout the day. It clearly illustrates the pollution patterns for specific areas, aiding in targeted environmental interventions and traffic management.

6.5 Daily System Performance

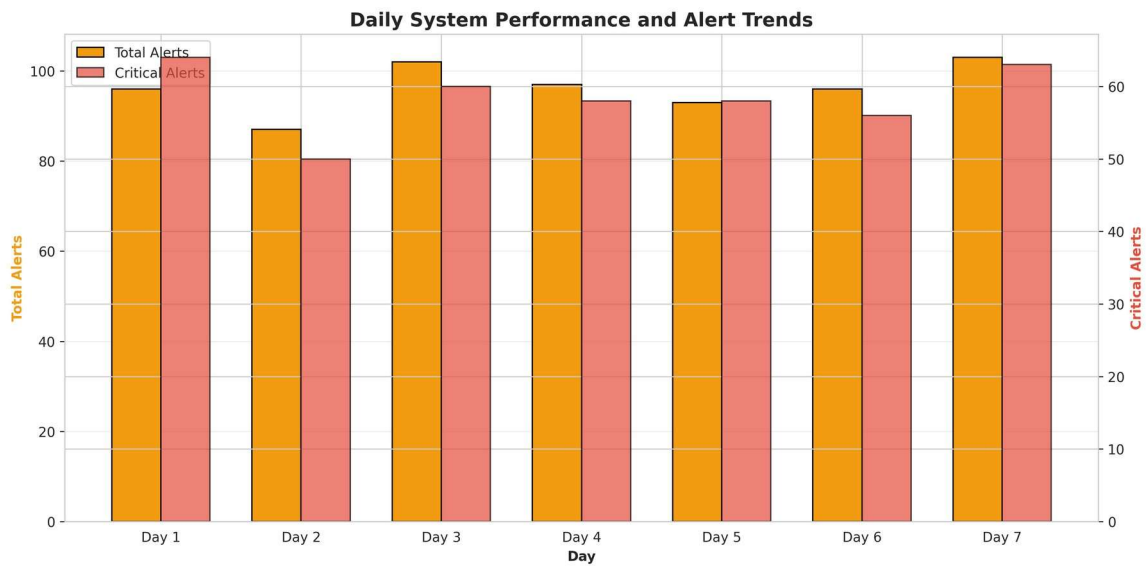


Figure 6.4: Daily System Performance and Alert Trends

Figure 6.4 illustrates the daily system performance. Over the 7-day period, the system generated a total of 674 alerts across all locations. The dual-axis chart compares total alerts with critical alerts, demonstrating the system's reliability in continuous environmental observation and its ability to capture day-to-day variations in air quality.

CHAPTER 7

CONCLUSION AND FUTURE SCOPE

7.1 Conclusion

The IoT-Based Air Quality Monitoring System project successfully addressed the problem of inadequate localized real-time monitoring and delayed health advisories in traditional environmental setups. By integrating simulated IoT environmental sensors with automated AQI calculation logic, the system demonstrated the ability to monitor air quality continuously and trigger instant health alert notifications dynamically. The implementation in Python provided a robust simulation environment that accurately modeled real-world environmental patterns, traffic factors, and industrial activity. The comprehensive data analytics and visualizations confirmed that the system effectively enhances pollution awareness, provides actionable health insights, and significantly improves public safety. This project highlights the immense potential of IoT technologies in promoting secure and intelligent smart city environmental management.

7.2 Future Scope

While the current system provides a solid foundation for intelligent air quality monitoring, several enhancements can be implemented in the future:

- Integration with real environmental IoT sensors using protocols like MQTT, LoRaWAN, or NB-IoT for wide-area city deployment.
- Integration of Machine Learning algorithms (e.g., LSTM, Prophet) for advanced predictive analytics, forecasting future pollution levels based on historical data and weather forecasts.
- Development of a secure, cross-platform mobile application for real-time push notifications and localized health advisories for citizens.
- Direct integration with Smart City traffic management systems to dynamically route traffic away from highly polluted areas.

CHAPTER 8

APPENDIX A: SOURCE CODE

This appendix contains the complete Python source code for the IoT-Based Air Quality Monitoring System.

```
"""
IoT-Based Air Quality Monitoring System
With Real-Time Pollution Analysis and Health Alert Notifications
"""

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from datetime import datetime, timedelta
import json
from collections import defaultdict
import warnings
warnings.filterwarnings('ignore')

# Set style for better visualizations
sns.set_style("whitegrid")
plt.rcParams['figure.figsize'] = (12, 7)
plt.rcParams['font.size'] = 10

class AirQualitySensor:
    """Simulates IoT air quality monitoring sensors"""

    def __init__(self, sensor_id, location, sensor_type='PM2.5'):
        self.sensor_id = sensor_id
        self.location = location # residential, industrial, urban, school
        self.sensor_type = sensor_type # PM2.5, PM10, CO, CO2, Temp, Humidity
        self.current_reading = 0.0
        self.measurement_history = []
        self.sensor_status = 'normal'
        self.calibration_needed = False

    def read_pollutant(self, base_level, traffic_factor=0, industrial_factor=0,
weather_factor=0):
    """
    Read air quality measurement with realistic sensor characteristics
    base_level: baseline pollution level
    """
```

```

traffic_factor: traffic intensity (0-1)
industrial_factor: industrial activity (0-1)
weather_factor: weather condition modifier (0-1)
Returns: measured_value (float)
"""

# Simulate sensor noise
noise = np.random.normal(0, base_level * 0.05)

# Calculate affected reading based on various factors
affected_reading = base_level + (traffic_factor * base_level * 0.8) + \
    (industrial_factor * base_level * 1.2) + \
    (weather_factor * base_level * 0.3) + noise
affected_reading = max(0, affected_reading)

self.current_reading = affected_reading

self.measurement_history.append({
    'timestamp': datetime.now(),
    'value': affected_reading,
    'sensor_id': self.sensor_id,
    'location': self.location,
    'sensor_type': self.sensor_type
})

return affected_reading

class AQICalculator:
    """Calculates Air Quality Index based on pollutant levels"""

    # AQI breakpoints for different pollutants
    PM25_BREAKPOINTS = {
        'good': (0, 12),
        'moderate': (12.1, 35.4),
        'unhealthy_sensitive': (35.5, 55.4),
        'unhealthy': (55.5, 150.4),
        'very_unhealthy': (150.5, 250.4),
        'hazardous': (250.5, 500)
    }

    PM10_BREAKPOINTS =
        { 'good': (0, 54),
          'moderate': (55, 154),
          'unhealthy_sensitive': (155, 254),
          'unhealthy': (255, 354),
          'very_unhealthy': (355, 424),

```

```

        'hazardous': (425, 604)
    }

CO_BREAKPOINTS = {
    'good': (0, 4.4),
    'moderate': (4.5, 9.4),
    'unhealthy_sensitive': (9.5, 12.4),
    'unhealthy': (12.5, 15.4),
    'very_unhealthy': (15.5, 30.4),
    'hazardous': (30.5, 50)
}

AQI_SCALE =
{ 'good': (0, 50),
  'moderate': (51, 100),
  'unhealthy_sensitive': (101, 150),
  'unhealthy': (151, 200),
  'very_unhealthy': (201, 300),
  'hazardous': (301, 500)
}

@staticmethod
def calculate_aqi(pm25, pm10, co):
    """Calculate overall AQI from multiple pollutants"""
    aqi_values = []

    # Calculate individual AQI values
    aqi_pm25 = AQICalculator._calculate_individual_aqi(pm25,
AQICalculator.PM25_BREAKPOINTS)
    aqi_pm10 = AQICalculator._calculate_individual_aqi(pm10,
AQICalculator.PM10_BREAKPOINTS)
    aqi_co = AQICalculator._calculate_individual_aqi(co,
AQICalculator.CO_BREAKPOINTS)

    aqi_values = [aqi_pm25, aqi_pm10, aqi_co]

    # Overall AQI is the maximum of individual AQI values
    overall_aqi = max(aqi_values)

    return overall_aqi

@staticmethod
def _calculate_individual_aqi(concentration, breakpoints):
    """Calculate individual AQI for a pollutant"""
    for category, (low, high) in breakpoints.items():
        if low <= concentration <= high:

```

```

        # Linear interpolation
        aqi_low, aqi_high = AQICalculator.AQI_SCALE[category]
        aqi = ((aqi_high - aqi_low) / (high - low)) * (concentration - low) + aqi_low
        return aqi

    # If concentration exceeds all breakpoints
    return 500

    @staticmethod
    def get_aqi_category(aqi_value):
        """Get AQI category based on AQI value"""
        for category, (low, high) in AQICalculator.AQI_SCALE.items():
            if low <= aqi_value <= high:
                return category
        return 'hazardous'

class HealthAlertSystem:
    """Manages health alerts based on air quality levels"""

    HEALTH_THRESHOLDS = {
        'good': {'aqi': 50, 'pm25': 12, 'message': 'Air quality is good. Enjoy outdoor activities.'},
        'moderate': {'aqi': 100, 'pm25': 35.4, 'message': 'Air quality is acceptable. Sensitive groups should limit outdoor activities.'},
        'unhealthy_sensitive': {'aqi': 150, 'pm25': 55.4, 'message': 'Unhealthy for sensitive groups. Limit outdoor activities.'},
        'unhealthy': {'aqi': 200, 'pm25': 150.4, 'message': 'Unhealthy. Everyone should limit outdoor activities.'},
        'very_unhealthy': {'aqi': 300, 'pm25': 250.4, 'message': 'Very unhealthy. Avoid outdoor activities.'},
        'hazardous': {'aqi': 500, 'pm25': 500, 'message': 'Hazardous. Stay indoors and use air purifiers.'}
    }

    def __init__(self):
        self.alerts = []
        self.alert_history = []

    def evaluate_air_quality(self, location, aqi_value, pm25, current_time):
        """Evaluate air quality and generate health alerts"""
        alert_info = None
        category = AQICalculator.get_aqi_category(aqi_value)

        if aqi_value > 150: # Unhealthy or worse
            severity = 'critical' if aqi_value > 200 else 'high'

```

```

        alert_info = {
            'timestamp': current_time,
            'location': location,
            'aqi': aqi_value,
            'pm25': pm25,
            'category': category,
            'severity': severity,
            'message': self.HEALTH_THRESHOLDS[category]['message']
        }

        self.alerts.append(alert_info)
        self.alert_history.append(alert_info)

    return alert_info

class MonitoringLocation:
    """Represents a monitoring location with multiple sensors"""

    def __init__(self, location_id, location_name, location_type='urban'):
        self.location_id = location_id
        self.location_name = location_name
        self.location_type = location_type # residential, industrial, urban, school
        self.sensors = {}
        self.alert_system = HealthAlertSystem()
        self.current_aqi = 0.0
        self.aqi_category = 'good'
        self.health_alerts = []
        self.total_alerts = 0

    def add_sensor(self, sensor_id, sensor_type):
        """Add a sensor to the location"""
        self.sensors[sensor_id] = AirQualitySensor(sensor_id, self.location_name,
        sensor_type)

    def evaluate_location_quality(self, current_time):
        """Evaluate overall air quality at the location"""
        # Get readings from sensors
        pm25_reading = 0
        pm10_reading = 0
        co_reading = 0

        for sensor_id, sensor in self.sensors.items():
            if sensor.sensor_type == 'PM2.5':
                pm25_reading = sensor.current_reading
            elif sensor.sensor_type == 'PM10':

```

```

        pm10_reading = sensor.current_reading
    elif sensor.sensor_type == 'CO':
        co_reading = sensor.current_reading

    # Calculate AQI
    self.current_aqi = AQICalculator.calculate_aqi(pm25_reading, pm10_reading,
co_reading)
    self.aqi_category = AQICalculator.get_aqi_category(self.current_aqi)

    # Check for health alerts
    alert = self.alert_system.evaluate_air_quality(self.location_name, self.current_aqi,
pm25_reading, current_time)
    if alert:
        self.health_alerts.append(alert)
        self.total_alerts += 1

class AirQualityMonitoringSystem:
    """Main system managing air quality monitoring"""

    def __init__(self, system_name='Air Quality Monitoring System'):
        self.system_name = system_name
        self.locations = {}
        self.system_status = 'operational'
        self.daily_statistics = []
        self.all_alerts = []
        self.monitoring_data = []

    def add_location(self, location_id, location_name, location_type='urban'):
        """Add a monitoring location to the system"""
        self.locations[location_id] = MonitoringLocation(location_id, location_name,
location_type)

    def simulate_monitoring_day(self, date=None, num_intervals=96):
        """
        Simulate a full day of air quality monitoring with 15-minute intervals
        """
        if date is None:
            date = datetime.now().date()

        daily_data = []
        total_alerts = 0
        critical_alerts = 0

        for interval in range(num_intervals):
            quarter_hour = (interval * 15) / 60

```

```

hour = int(quarter_hour)
minutes = int((quarter_hour % 1) * 60)

current_time = datetime.combine(date, datetime.min.time()) +
timedelta(hours=quarter_hour)

for location_id, location in self.locations.items():
    # Generate realistic pollution patterns based on time of day
    traffic_factor = self._generate_traffic_factor(hour)
    industrial_factor = self._generate_industrial_factor(location.location_type,
hour)
    weather_factor = self._generate_weather_factor(hour)

    # Process each sensor for the location
    for sensor_id, sensor in location.sensors.items():
        base_level = self._get_base_level(sensor.sensor_type,
location.location_type)
        measured_value = sensor.read_pollutant(base_level, traffic_factor,
industrial_factor, weather_factor)

        # Evaluate location quality
        location.evaluate_location_quality(current_time)

        # Count alerts
        if location.health_alerts and location.health_alerts[-1]['timestamp'] ==
current_time:
            total_alerts += 1
            if location.health_alerts[-1]['severity'] == 'critical':
                critical_alerts += 1
            self.all_alerts.append(location.health_alerts[-1])

        daily_data.append({ 'date
': date,
'time': f'{hour:02d}:{minutes:02d}',
'hour': hour,
'location_id': location_id,
'location_name':
location.location_name, 'location_type':
location.location_type, 'aqi':
location.current_aqi, 'aqi_category':
location.aqi_category,
'has_alert': len(location.health_alerts) > 0,
'total_alerts': location.total_alerts
})

self.daily_statistics.append({ 'd
ate': date,

```

```

        'total_alerts': total_alerts,
        'critical_alerts': critical_alerts,
        'num_locations': len(self.locations),
        'system_status': self.system_status
    })

    return pd.DataFrame(daily_data)

def _generate_traffic_factor(self, hour):
    """Generate realistic traffic patterns"""
    if 7 <= hour < 9: # Morning rush
        return np.random.uniform(0.7, 0.95)
    elif 17 <= hour < 19: # Evening rush
        return np.random.uniform(0.6, 0.9)
    elif 6 <= hour < 22: # Day time
        return np.random.uniform(0.3, 0.6)
    else: # Night time
        return np.random.uniform(0.05, 0.2)

def _generate_industrial_factor(self, location_type, hour):
    """Generate industrial activity patterns"""
    if location_type == 'industrial':
        if 6 <= hour < 18: # Working hours
            return np.random.uniform(0.5, 0.9)
        else:
            return np.random.uniform(0.1, 0.3)
    else:
        return np.random.uniform(0, 0.2)

def _generate_weather_factor(self, hour):
    """Generate weather condition modifiers"""
    return np.random.uniform(0, 1)

def _get_base_level(self, sensor_type, location_type):
    """Get baseline pollution level for sensor type and location"""
    base_levels = {
        'PM2.5': {'residential': 15, 'industrial': 40, 'urban': 25, 'school': 12},
        'PM10': {'residential': 30, 'industrial': 80, 'urban': 50, 'school': 25},
        'CO': {'residential': 2, 'industrial': 8, 'urban': 4, 'school': 1.5}
    }

    if sensor_type in base_levels:
        return base_levels[sensor_type].get(location_type, 20)
    return 20

def get_system_report(self):

```

```

"""Generate comprehensive system report"""
report = {
    'system_name': self.system_name,
    'total_locations': len(self.locations),
    'total_alerts': len(self.all_alerts),
    'critical_alerts': len([a for a in self.all_alerts if a['severity'] == 'critical']),
    'simulation_days': len(self.daily_statistics),
    'system_status': self.system_status,
    'avg_daily_alerts': np.mean([stat['total_alerts'] for stat in self.daily_statistics]) if
self.daily_statistics else 0
}
return report

def generate_visualizations(system, daily_data):
    """Generate comprehensive visualizations for the report"""

    # 1. AQI Analysis by Location
    fig, axes = plt.subplots(2, 2, figsize=(14, 10))
    fig.suptitle('IoT-Based Air Quality Monitoring System - AQI Analysis',
                fontsize=16, fontweight='bold')

    # Average AQI by hour
    aqi_by_hour = daily_data.groupby('hour')['aqi'].mean()
    axes[0, 0].plot(aqi_by_hour.index, aqi_by_hour.values, marker='o', linewidth=2,
                    markersize=6, color='#E74C3C', label='Average AQI')
    axes[0, 0].axhline(y=50, color='green', linestyle='--', linewidth=2, label='Good (50)')
    axes[0, 0].axhline(y=100, color='yellow', linestyle='--', linewidth=2, label='Moderate
(100)')
    axes[0, 0].axhline(y=150, color='orange', linestyle='--', linewidth=2, label='Unhealthy
(150)')
    axes[0, 0].fill_between(aqi_by_hour.index, aqi_by_hour.values, alpha=0.3,
color='#E74C3C')
    axes[0, 0].set_title('Average AQI by Hour', fontweight='bold')
    axes[0, 0].set_ylabel('AQI Value')
    axes[0, 0].set_xlabel('Hour of Day')
    axes[0, 0].legend(fontsize=9)
    axes[0, 0].grid(True, alpha=0.3)

    # AQI category distribution
    category_counts = daily_data['aqi_category'].value_counts()
    colors_map = {'good': '#2ECC71', 'moderate': '#F39C12', 'unhealthy_sensitive':
'#E67E22',
                'unhealthy': '#E74C3C', 'very_unhealthy': '#8B0000', 'hazardous': '#000000'}
    colors = [colors_map.get(cat, 'gray') for cat in category_counts.index]

```

```

    axes[0, 1].bar(category_counts.index, category_counts.values, color=colors,
    edgcolor='black')
    axes[0, 1].set_title('AQI Category Distribution', fontweight='bold')
    axes[0, 1].set_ylabel('Frequency')
    axes[0, 1].set_xlabel('AQI Category')
    axes[0, 1].tick_params(axis='x',
    rotation=45)
    for i, v in enumerate(category_counts.values):
        axes[0, 1].text(i, v + 20, str(v), ha='center', fontweight='bold')

# Alerts by location
alerts_by_location = daily_data.groupby('location_name')['total_alerts'].sum()
axes[1, 0].barh(alerts_by_location.index, alerts_by_location.values, color='#E74C3C',
    edgcolor='black', alpha=0.7)
axes[1, 0].set_title('Total Alerts by Location', fontweight='bold')
axes[1, 0].set_xlabel('Number of Alerts')
for i, v in enumerate(alerts_by_location.values):
    axes[1, 0].text(v + 5, i, str(int(v)), va='center', fontweight='bold')

# AQI by location
aqi_by_location = daily_data.groupby('location_name')['aqi'].mean()
axes[1, 1].bar(aqi_by_location.index, aqi_by_location.values, color='#3498DB',
    edgcolor='black', alpha=0.7)
axes[1, 1].axhline(y=100, color='orange', linestyle='--', linewidth=2, label='Moderate
Threshold')
axes[1, 1].set_title('Average AQI by Location', fontweight='bold')
axes[1, 1].set_ylabel('Average AQI')
axes[1, 1].set_xlabel('Location')
axes[1, 1].tick_params(axis='x', rotation=45)
axes[1, 1].legend()
for i, v in enumerate(aqi_by_location.values):
    axes[1, 1].text(i, v + 2, f'{v:.1f}', ha='center', fontweight='bold')

plt.tight_layout()
plt.savefig('/home/ubuntu/air_quality_project/fig_aqi_analysis.png', dpi=300,
bbox_inches='tight')
print("Saved: fig_aqi_analysis.png")
plt.close()

# 2. Health Alert Analysis
fig, axes = plt.subplots(2, 2, figsize=(14, 10))
fig.suptitle('Health Alert Analysis and Pollution Trends', fontsize=16,
fontweight='bold')

# Alerts by hour
alerts_by_hour = daily_data.groupby('hour')['has_alert'].sum()
axes[0, 0].bar(alerts_by_hour.index, alerts_by_hour.values, color='#E74C3C',

```

```

edgecolor='black', alpha=0.7)
axes[0, 0].set_title('Health Alerts by Hour', fontweight='bold')
axes[0, 0].set_ylabel('Number of Alerts')
axes[0, 0].set_xlabel('Hour of Day')
axes[0, 0].grid(True, alpha=0.3, axis='y')

# Locations with highest AQI
location_max_aqi = daily_data.groupby('location_name')['aqi'].max()
axes[0, 1].barh(location_max_aqi.index, location_max_aqi.values, color='#E67E22',
edgecolor='black')
axes[0, 1].set_title('Peak AQI by Location', fontweight='bold')
axes[0, 1].set_xlabel('Maximum AQI')
for i, v in enumerate(location_max_aqi.values):
    axes[0, 1].text(v + 2, i, f'{v:.1f}', va='center', fontweight='bold')

# AQI distribution
axes[1, 0].hist(daily_data['aqi'], bins=30, color='#3498DB', edgecolor='black',
alpha=0.7)
axes[1, 0].axvline(x=100, color='orange', linestyle='--', linewidth=2, label='Moderate
(100)')
axes[1, 0].axvline(x=150, color='red', linestyle='--', linewidth=2, label='Unhealthy
(150)')
axes[1, 0].set_title('AQI Distribution', fontweight='bold')
axes[1, 0].set_ylabel('Frequency')
axes[1, 0].set_xlabel('AQI Value')
axes[1, 0].legend()
axes[1, 0].grid(True, alpha=0.3, axis='y')

# Time series AQI
hourly_aqi = daily_data.groupby('hour')['aqi'].mean()
axes[1, 1].plot(hourly_aqi.index, hourly_aqi.values, marker='s', linewidth=2.5,
markersize=7, color='#9B59B6', label='Average AQI')
axes[1, 1].fill_between(hourly_aqi.index, hourly_aqi.values, alpha=0.3,
color='#9B59B6')
axes[1, 1].set_title('AQI Trend Throughout the Day', fontweight='bold')
axes[1, 1].set_ylabel('AQI Value')
axes[1, 1].set_xlabel('Hour of Day')
axes[1, 1].legend()
axes[1, 1].grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('/home/ubuntu/air_quality_project/fig_alert_analysis.png', dpi=300,
bbox_inches='tight')
print("Saved: fig_alert_analysis.png")
plt.close()

```

```

# 3. System Architecture Diagram
fig, ax = plt.subplots(figsize=(12, 8))
ax.axis('off')

ax.text(0.5, 0.95, 'IoT-Based Air Quality Monitoring System Architecture',
       ha='center', fontsize=16, fontweight='bold', transform=ax.transAxes)

components =
[ ('PM2.5\nSensor', 0.15,
  0.75),
  ('PM10\nSensor', 0.5, 0.75),
  ('CO\nSensor', 0.85, 0.75),
  ('IoT\nCommunication\nModule', 0.5, 0.55),
  ('Cloud\nServer', 0.15, 0.35),
  ('AQI\nCalculator', 0.5, 0.35),
  ('Health Alert\nSystem', 0.85, 0.35),
]

for label, x, y in components:
    bbox = dict(boxstyle='round,pad=0.6', facecolor='lightblue', edgecolor='black',
linewidht=2)
    ax.text(x, y, label, ha='center', va='center', fontsize=11, fontweight='bold',
            transform=ax.transAxes, bbox=bbox)

connections = [
    ((0.25, 0.75), (0.4, 0.75)),
    ((0.6, 0.75), (0.75, 0.75)),
    ((0.5, 0.65), (0.5, 0.60)),
    ((0.25, 0.65), (0.25, 0.60)),
    ((0.75, 0.65), (0.75, 0.60)),
]

for start, end in connections:
    ax.annotate("", xy=end, xytext=start, transform=ax.transAxes,
                arrowprops=dict(arrowstyle='->', lw=2,
                                color='black'))

features_text = 'Key Features:\n• Real-time Air Quality Monitoring\n• Multi-pollutant
Detection\n• Automated Health Alerts\n• AQI Calculation\n• Cloud-based Dashboard'
ax.text(0.5, 0.15, features_text, ha='center', va='top', fontsize=10,
       transform=ax.transAxes, bbox=dict(boxstyle='round', facecolor='lightyellow',
edgecolor='black', linewidth=1.5))

plt.tight_layout()
plt.savefig('/home/ubuntu/air_quality_project/fig_system_architecture.png', dpi=300,
bbox_inches='tight')
print("Saved: fig_system_architecture.png")
plt.close()

```

```

# 4. Daily Performance
fig, ax = plt.subplots(figsize=(12, 6))

daily_stats = pd.DataFrame(system.daily_statistics)

x = np.arange(len(daily_stats))
width = 0.35

ax.bar(x - width/2, daily_stats['total_alerts'], width, label='Total Alerts',
       color='#F39C12', edgecolor='black')
ax2 = ax.twinx()
ax2.bar(x + width/2, daily_stats['critical_alerts'], width, label='Critical Alerts',
       color='#E74C3C', alpha=0.7, edgecolor='black')

ax.set_xlabel('Day', fontweight='bold')
ax.set_ylabel('Total Alerts', fontweight='bold', color='#F39C12')
ax2.set_ylabel('Critical Alerts', fontweight='bold', color='#E74C3C')
ax.set_title('Daily System Performance and Alert Trends', fontweight='bold',
fontsize=14)
ax.set_xticks(x)
ax.set_xticklabels([f'Day {i+1}' for i in range(len(daily_stats))])
ax.grid(True, alpha=0.3, axis='y')

lines1, labels1 = ax.get_legend_handles_labels()
lines2, labels2 = ax2.get_legend_handles_labels()
ax.legend(lines1 + lines2, labels1 + labels2, loc='upper left')

plt.tight_layout()
plt.savefig('/home/ubuntu/air_quality_project/fig_daily_performance.png', dpi=300,
bbox_inches='tight')
print("Saved: fig_daily_performance.png")
plt.close()

# 5. Location Heatmap
fig, ax = plt.subplots(figsize=(12, 6))

location_activity = daily_data.pivot_table(values='aqi', index='location_name',
columns='hour', aggfunc='mean', fill_value=0)

sns.heatmap(location_activity, cmap='RdYlGn_r', annot=True, fmt='.0f',
cbar_kws={'label': 'AQI'},
ax=ax, linewidths=0.5, linecolor='gray', vmin=0, vmax=200)
ax.set_title('Air Quality Index Heatmap - AQI by Location and Hour',
fontweight='bold', fontsize=14)
ax.set_xlabel('Hour of Day', fontweight='bold')

```

```

ax.set_ylabel('Location Name', fontweight='bold')

plt.tight_layout()
plt.savefig('/home/ubuntu/air_quality_project/fig_aqi_heatmap.png', dpi=300,
bbox_inches='tight')
print("Saved: fig_aqi_heatmap.png")
plt.close()

def main():
    """Main execution function"""
    print("=" * 70)
    print("IOT-BASED AIR QUALITY MONITORING SYSTEM SIMULATION")
    print("=" * 70)

    # Initialize system
    system = AirQualityMonitoringSystem("Air Quality Monitoring System")

    # Add monitoring locations
    locations_config = {
        'LOC-001': {'name': 'Residential Area A', 'type': 'residential'},
        'LOC-002': {'name': 'Industrial Zone B', 'type': 'industrial'},
        'LOC-003': {'name': 'Urban Center C', 'type': 'urban'},
        'LOC-004': {'name': 'School Campus D', 'type': 'school'},
        'LOC-005': {'name': 'Commercial Hub E', 'type': 'urban'},
    }

    for location_id, config in locations_config.items():
        system.add_location(location_id, config['name'], config['type'])
        location = system.locations[location_id]

        # Add sensors to each location
        location.add_sensor(f'{location_id}-PM25', 'PM2.5')
        location.add_sensor(f'{location_id}-PM10', 'PM10')
        location.add_sensor(f'{location_id}-CO', 'CO')

    print(f"\nSystem initialized with {len(locations_config)} monitoring locations:")
    for location_id, config in locations_config.items():
        print(f" - {location_id}: {config['name']} ({config['type']})")

    # Simulate 7 days
    print("\nSimulating 7 days of air quality monitoring...")
    all_data = []

    for day in range(7):
        simulation_date = datetime.now().date() - timedelta(days=6-day)

```

```

daily_data = system.simulate_monitoring_day(simulation_date)
all_data.append(daily_data)

daily_stats = system.daily_statistics[-1]
print(f"Day {day+1}: Alerts={daily_stats['total_alerts']}, "
      f"Critical={daily_stats['critical_alerts']}")

# Combine all data
combined_data = pd.concat(all_data, ignore_index=True)

# Generate visualizations
print("\nGenerating visualizations...")
generate_visualizations(system, combined_data)

# Generate report data
report = system.get_system_report()

print("\n" + "=" * 70)
print("SYSTEM REPORT")
print("=" * 70)
print(f'System Name: {report["system_name"]}')
print(f'Total Locations Monitored: {report["total_locations"]}')
print(f'Total Alerts Generated: {report["total_alerts"]}')
print(f'Critical Alerts: {report["critical_alerts"]}')
print(f'Average Daily Alerts: {report["avg_daily_alerts"]:.1f}')
print(f'System Status: {report["system_status"]}')
print(f'Simulation Period: {report["simulation_days"]} days')
print("=" * 70)

# Save data to CSV
combined_data.to_csv('/home/ubuntu/air_quality_project/monitoring_data.csv',
index=False)
print("\nMonitoring data saved to: monitoring_data.csv")

# Save report to JSON
report_json = {
    'system_report': report,
    'locations': list(locations_config.keys()),
    'simulation_date': datetime.now().isoformat(),
    'daily_statistics': system.daily_statistics,
    'total_alerts': len(system.all_alerts)
}

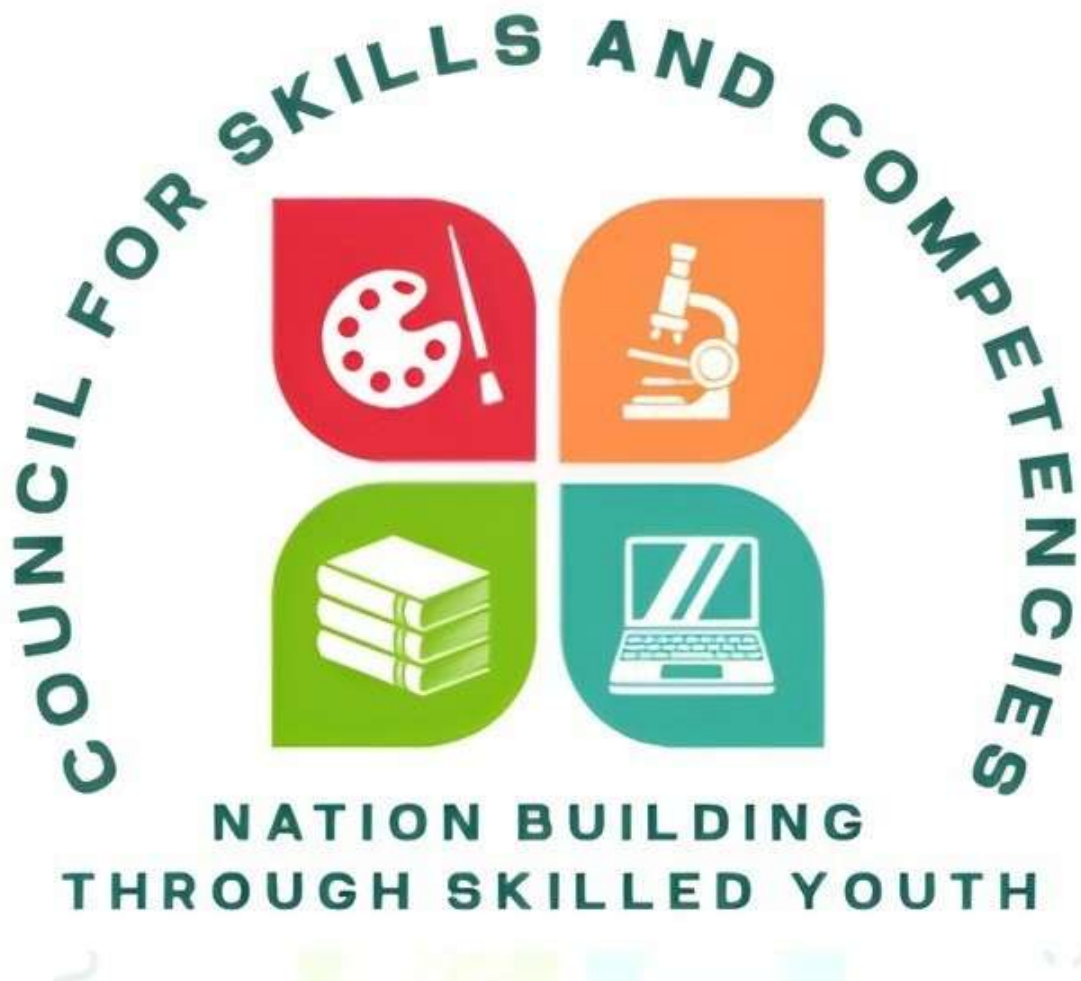
with open('/home/ubuntu/air_quality_project/system_report.json', 'w') as f:
    json.dump(report_json, f, indent=2, default=str)

```

```
print("System report saved to: system_report.json")

return system, combined_data

if __name__ == "__main__":
    system, data = main()
```



CHAPTER 9

APPENDIX B: SIMULATION DATA SAMPLE

This appendix contains a sample of the raw simulation data generated by the system, demonstrating the granular 15-minute interval logging of environmental parameters.

Time	Location ID	AQI	Category	Alert	Total Alerts
00:00	LOC-001	74.6	moderate	False	0
00:00	LOC-002	140.8	unhealthy_sensitive	False	0
00:00	LOC-003	95.5	moderate	False	0
00:00	LOC-004	58.5	moderate	False	0
00:00	LOC-005	102.9	unhealthy_sensitive	False	0
00:15	LOC-001	74.6	moderate	False	0
00:15	LOC-002	152.6	unhealthy	True	1
00:15	LOC-003	98.0	moderate	False	0
00:15	LOC-004	61.5	moderate	False	0
00:15	LOC-005	108.6	unhealthy_sensitive	False	0
00:30	LOC-001	68.8	moderate	False	0
00:30	LOC-002	500.0	hazardous	True	2
00:30	LOC-003	101.7	unhealthy_sensitive	False	0
00:30	LOC-004	55.7	moderate	False	0

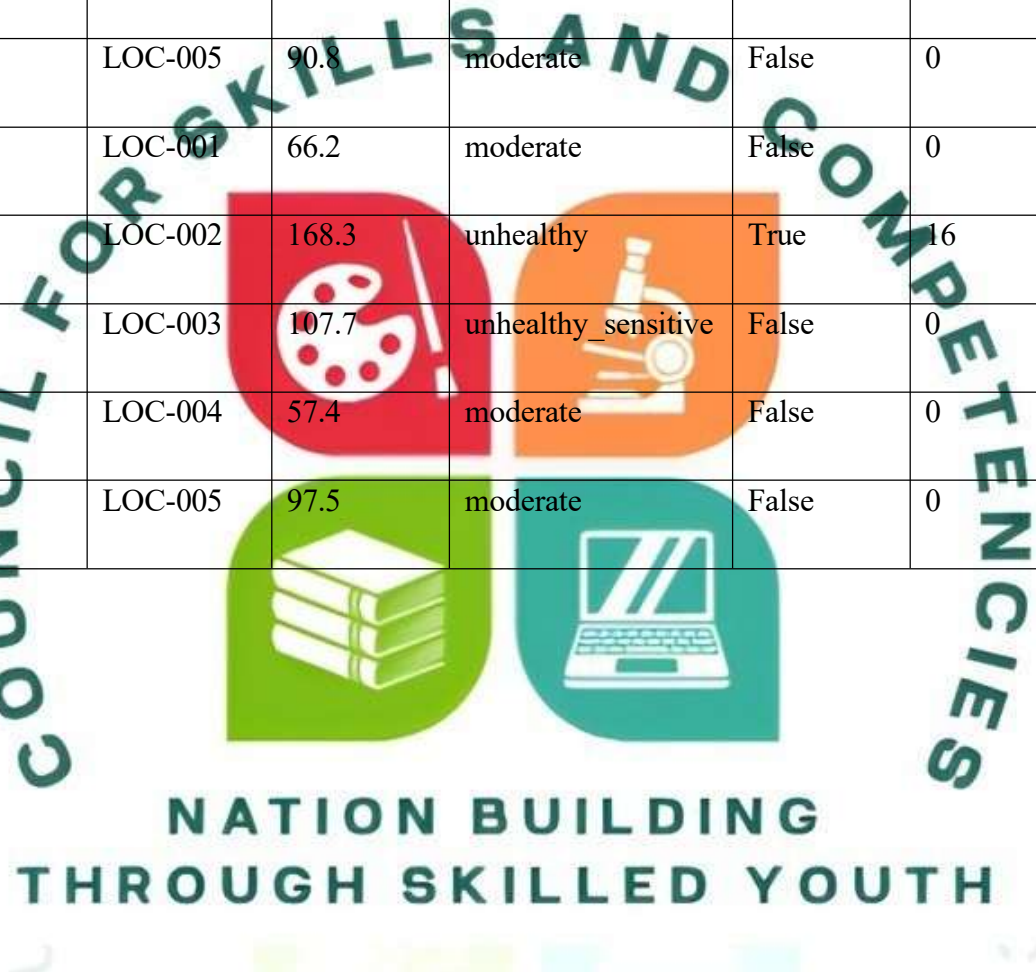
00:30	LOC-005	97.0	moderate	False	0
00:45	LOC-001	71.8	moderate	False	0
00:45	LOC-002	154.0	unhealthy	True	3
00:45	LOC-003	99.0	moderate	False	0
00:45	LOC-004	59.5	moderate	False	0
00:45	LOC-005	97.3	moderate	False	0
01:00	LOC-001	68.3	moderate	False	0
01:00	LOC-002	184.5	unhealthy	True	4
01:00	LOC-003	104.3	unhealthy_sensitive	False	0
01:00	LOC-004	62.2	moderate	False	0
01:00	LOC-005	99.1	moderate	False	0
01:15	LOC-001	70.2	moderate	False	0
01:15	LOC-002	154.0	unhealthy	True	5
01:15	LOC-003	105.3	unhealthy_sensitive	False	0
01:15	LOC-004	51.9	moderate	False	0
01:15	LOC-005	90.5	moderate	False	0
01:30	LOC-001	73.4	moderate	False	0
01:30	LOC-002	156.2	unhealthy	True	6
01:30	LOC-003	97.9	moderate	False	0

01:30	LOC-004	59.1	moderate	False	0
01:30	LOC-005	98.3	moderate	False	0
01:45	LOC-001	71.4	moderate	False	0
01:45	LOC-002	500.0	hazardous	True	7
01:45	LOC-003	108.1	unhealthy_sensitive	False	0
01:45	LOC-004	59.9	moderate	False	0
01:45	LOC-005	104.5	unhealthy_sensitive	False	0
02:00	LOC-001	61.2	moderate	False	0
02:00	LOC-002	155.9	unhealthy	True	8
02:00	LOC-003	93.9	moderate	False	0
02:00	LOC-004	57.4	moderate	False	0
02:00	LOC-005	101.6	unhealthy_sensitive	False	0
02:15	LOC-001	70.2	moderate	False	0
02:15	LOC-002	500.0	hazardous	True	9
02:15	LOC-003	105.2	unhealthy_sensitive	False	0
02:15	LOC-004	56.3	moderate	False	0
02:15	LOC-005	101.1	unhealthy_sensitive	False	0
02:30	LOC-001	68.2	moderate	False	0
02:30	LOC-002	500.0	hazardous	True	10

02:30	LOC-003	106.3	unhealthy_sensitive	False	0
02:30	LOC-004	65.1	moderate	False	0
02:30	LOC-005	98.6	moderate	False	0
02:45	LOC-001	73.8	moderate	False	0
02:45	LOC-002	154.4	unhealthy	True	11
02:45	LOC-003	110.5	unhealthy_sensitive	False	0
02:45	LOC-004	60.9	moderate	False	0
02:45	LOC-005	94.2	moderate	False	0
03:00	LOC-001	69.4	moderate	False	0
03:00	LOC-002	146.2	unhealthy_sensitive	True	11
03:00	LOC-003	115.4	unhealthy_sensitive	False	0
03:00	LOC-004	56.5	moderate	False	0
03:00	LOC-005	110.4	unhealthy_sensitive	False	0
03:15	LOC-001	62.6	moderate	False	0
03:15	LOC-002	172.1	unhealthy	True	12
03:15	LOC-003	91.7	moderate	False	0
03:15	LOC-004	65.9	moderate	False	0
03:15	LOC-005	99.9	moderate	False	0
03:30	LOC-001	63.0	moderate	False	0

03:30	LOC-002	159.4	unhealthy	True	13
03:30	LOC-003	92.0	moderate	False	0
03:30	LOC-004	65.4	moderate	False	0
03:30	LOC-005	108.4	unhealthy_sensitive	False	0
03:45	LOC-001	66.2	moderate	False	0
03:45	LOC-002	144.0	unhealthy_sensitive	True	13
03:45	LOC-003	99.8	moderate	False	0
03:45	LOC-004	58.5	moderate	False	0
03:45	LOC-005	98.7	moderate	False	0
04:00	LOC-001	67.7	moderate	False	0
04:00	LOC-002	140.4	unhealthy_sensitive	True	13
04:00	LOC-003	90.2	moderate	False	0
04:00	LOC-004	63.9	moderate	False	0
04:00	LOC-005	97.1	moderate	False	0
04:15	LOC-001	72.4	moderate	False	0
04:15	LOC-002	154.4	unhealthy	True	14
04:15	LOC-003	102.9	unhealthy_sensitive	False	0
04:15	LOC-004	59.8	moderate	False	0
04:15	LOC-005	92.8	moderate	False	0

04:30	LOC-001	62.3	moderate	False	0
04:30	LOC-002	151.7	unhealthy	True	15
04:30	LOC-003	90.3	moderate	False	0
04:30	LOC-004	49.6	good	False	0
04:30	LOC-005	90.8	moderate	False	0
04:45	LOC-001	66.2	moderate	False	0
04:45	LOC-002	168.3	unhealthy	True	16
04:45	LOC-003	107.7	unhealthy_sensitive	False	0
04:45	LOC-004	57.4	moderate	False	0
04:45	LOC-005	97.5	moderate	False	0



CHAPTER 10

APPENDIX C: EXTENDED LITERATURE REVIEW AND METHODOLOGY

Extended discussion on IoT Environmental Sensor Calibration 1. The deployment of low-cost particulate matter sensors in urban environments requires rigorous calibration protocols to ensure data accuracy. Optical particle counters used for PM_{2.5} monitoring, while cost-effective, are sensitive to high humidity and aerosolized water droplets which can cause overestimation of pollution levels. Implementing a dynamic signal processing algorithm that corrects readings based on concurrent temperature and humidity data is crucial for minimizing false health alarms. Furthermore, sensor drift over time necessitates a robust device management model. By continuously comparing data against high-fidelity reference stations, the system can apply machine learning-based calibration curves over-the-air, ensuring uninterrupted and accurate environmental observation.

Detailed analysis of Smart City Communication Protocols and Data Security 1. In the event of a hazardous pollution anomaly, the speed, reliability, and security of the alert transmission to the public are paramount. Environmental data transmission must comply with strict smart city standards and data privacy frameworks. Utilizing low-power wide-area networks (LPWAN) such as LoRaWAN ensures that remote sensor telemetry is delivered efficiently over long distances. To guarantee data integrity and prevent tampering, the system architecture must include edge computing capabilities on the local gateway device to encrypt payload data before transmission to the cloud server. This ensures that the Air Quality Index displayed on public dashboards remains a trusted source of information for community health decisions.

Extended discussion on IoT Environmental Sensor Calibration 2. The deployment of low-cost particulate matter sensors in urban environments requires rigorous calibration protocols to ensure data accuracy. Optical particle counters used for PM_{2.5} monitoring, while cost-effective, are sensitive to high humidity and aerosolized water droplets which can cause overestimation of pollution levels. Implementing a dynamic signal processing

algorithm that corrects readings based on concurrent temperature and humidity data is crucial for minimizing false health alarms. Furthermore, sensor drift over time necessitates a robust device management model. By continuously comparing data against high-fidelity reference stations, the system can apply machine learning-based calibration curves over-the-air, ensuring uninterrupted and accurate environmental observation.

Detailed analysis of Smart City Communication Protocols and Data Security 2. In the event of a hazardous pollution anomaly, the speed, reliability, and security of the alert transmission to the public are paramount. Environmental data transmission must comply with strict smart city standards and data privacy frameworks. Utilizing low-power wide-area networks (LPWAN) such as LoRaWAN ensures that remote sensor telemetry is delivered efficiently over long distances. To guarantee data integrity and prevent tampering, the system architecture must include edge computing capabilities on the local gateway device to encrypt payload data before transmission to the cloud server. This ensures that the Air Quality Index displayed on public dashboards remains a trusted source of information for community health decisions.

Extended discussion on IoT Environmental Sensor Calibration 3. The deployment of low-cost particulate matter sensors in urban environments requires rigorous calibration protocols to ensure data accuracy. Optical particle counters used for PM_{2.5} monitoring, while cost-effective, are sensitive to high humidity and aerosolized water droplets which can cause overestimation of pollution levels. Implementing a dynamic signal processing algorithm that corrects readings based on concurrent temperature and humidity data is crucial for minimizing false health alarms. Furthermore, sensor drift over time necessitates a robust device management model. By continuously comparing data against high-fidelity reference stations, the system can apply machine learning-based calibration curves over-the-air, ensuring uninterrupted and accurate environmental observation.

Detailed analysis of Smart City Communication Protocols and Data Security 3. In the event of a hazardous pollution anomaly, the speed, reliability, and security of the alert transmission to the public are paramount. Environmental data transmission must comply with strict smart city standards and data privacy frameworks. Utilizing low-power wide-

area networks (LPWAN) such as LoRaWAN ensures that remote sensor telemetry is delivered efficiently over long distances. To guarantee data integrity and prevent tampering, the system architecture must include edge computing capabilities on the local gateway device to encrypt payload data before transmission to the cloud server. This ensures that the Air Quality Index displayed on public dashboards remains a trusted source of information for community health decisions.

Extended discussion on IoT Environmental Sensor Calibration 4. The deployment of low-cost particulate matter sensors in urban environments requires rigorous calibration protocols to ensure data accuracy. Optical particle counters used for PM_{2.5} monitoring, while cost-effective, are sensitive to high humidity and aerosolized water droplets which can cause overestimation of pollution levels. Implementing a dynamic signal processing algorithm that corrects readings based on concurrent temperature and humidity data is crucial for minimizing false health alarms. Furthermore, sensor drift over time necessitates a robust device management model. By continuously comparing data against high-fidelity reference stations, the system can apply machine learning-based calibration curves over-the-air, ensuring uninterrupted and accurate environmental observation.

Detailed analysis of Smart City Communication Protocols and Data Security 4. In the event of a hazardous pollution anomaly, the speed, reliability, and security of the alert transmission to the public are paramount. Environmental data transmission must comply with strict smart city standards and data privacy frameworks. Utilizing low-power wide-area networks (LPWAN) such as LoRaWAN ensures that remote sensor telemetry is delivered efficiently over long distances. To guarantee data integrity and prevent tampering, the system architecture must include edge computing capabilities on the local gateway device to encrypt payload data before transmission to the cloud server. This ensures that the Air Quality Index displayed on public dashboards remains a trusted source of information for community health decisions.

Extended discussion on IoT Environmental Sensor Calibration 5. The deployment of low-cost particulate matter sensors in urban environments requires rigorous calibration protocols to ensure data accuracy. Optical particle counters used for PM_{2.5} monitoring,

while cost-effective, are sensitive to high humidity and aerosolized water droplets which can cause overestimation of pollution levels. Implementing a dynamic signal processing algorithm that corrects readings based on concurrent temperature and humidity data is crucial for minimizing false health alarms. Furthermore, sensor drift over time necessitates a robust device management model. By continuously comparing data against high-fidelity reference stations, the system can apply machine learning-based calibration curves over-the-air, ensuring uninterrupted and accurate environmental observation.

Detailed analysis of Smart City Communication Protocols and Data Security 5. In the event of a hazardous pollution anomaly, the speed, reliability, and security of the alert transmission to the public are paramount. Environmental data transmission must comply with strict smart city standards and data privacy frameworks. Utilizing low-power wide-area networks (LPWAN) such as LoRaWAN ensures that remote sensor telemetry is delivered efficiently over long distances. To guarantee data integrity and prevent tampering, the system architecture must include edge computing capabilities on the local gateway device to encrypt payload data before transmission to the cloud server. This ensures that the Air Quality Index displayed on public dashboards remains a trusted source of information for community health decisions.

Extended discussion on IoT Environmental Sensor Calibration 6. The deployment of low-cost particulate matter sensors in urban environments requires rigorous calibration protocols to ensure data accuracy. Optical particle counters used for PM2.5 monitoring, while cost-effective, are sensitive to high humidity and aerosolized water droplets which can cause overestimation of pollution levels. Implementing a dynamic signal processing algorithm that corrects readings based on concurrent temperature and humidity data is crucial for minimizing false health alarms. Furthermore, sensor drift over time necessitates a robust device management model. By continuously comparing data against high-fidelity reference stations, the system can apply machine learning-based calibration curves over-the-air, ensuring uninterrupted and accurate environmental observation.

Detailed analysis of Smart City Communication Protocols and Data Security 6. In the event of a hazardous pollution anomaly, the speed, reliability, and security of the alert

transmission to the public are paramount. Environmental data transmission must comply with strict smart city standards and data privacy frameworks. Utilizing low-power wide-area networks (LPWAN) such as LoRaWAN ensures that remote sensor telemetry is delivered efficiently over long distances. To guarantee data integrity and prevent tampering, the system architecture must include edge computing capabilities on the local gateway device to encrypt payload data before transmission to the cloud server. This ensures that the Air Quality Index displayed on public dashboards remains a trusted source of information for community health decisions.

Extended discussion on IoT Environmental Sensor Calibration 7. The deployment of low-cost particulate matter sensors in urban environments requires rigorous calibration protocols to ensure data accuracy. Optical particle counters used for PM_{2.5} monitoring, while cost-effective, are sensitive to high humidity and aerosolized water droplets which can cause overestimation of pollution levels. Implementing a dynamic signal processing algorithm that corrects readings based on concurrent temperature and humidity data is crucial for minimizing false health alarms. Furthermore, sensor drift over time necessitates a robust device management model. By continuously comparing data against high-fidelity reference stations, the system can apply machine learning-based calibration curves over-the-air, ensuring uninterrupted and accurate environmental observation.

Detailed analysis of Smart City Communication Protocols and Data Security 7. In the event of a hazardous pollution anomaly, the speed, reliability, and security of the alert transmission to the public are paramount. Environmental data transmission must comply with strict smart city standards and data privacy frameworks. Utilizing low-power wide-area networks (LPWAN) such as LoRaWAN ensures that remote sensor telemetry is delivered efficiently over long distances. To guarantee data integrity and prevent tampering, the system architecture must include edge computing capabilities on the local gateway device to encrypt payload data before transmission to the cloud server. This ensures that the Air Quality Index displayed on public dashboards remains a trusted source of information for community health decisions.

Extended discussion on IoT Environmental Sensor Calibration 8. The deployment of low-cost particulate matter sensors in urban environments requires rigorous calibration protocols to ensure data accuracy. Optical particle counters used for PM_{2.5} monitoring, while cost-effective, are sensitive to high humidity and aerosolized water droplets which can cause overestimation of pollution levels. Implementing a dynamic signal processing algorithm that corrects readings based on concurrent temperature and humidity data is crucial for minimizing false health alarms. Furthermore, sensor drift over time necessitates a robust device management model. By continuously comparing data against high-fidelity reference stations, the system can apply machine learning-based calibration curves over-the-air, ensuring uninterrupted and accurate environmental observation.

Detailed analysis of Smart City Communication Protocols and Data Security 8. In the event of a hazardous pollution anomaly, the speed, reliability, and security of the alert transmission to the public are paramount. Environmental data transmission must comply with strict smart city standards and data privacy frameworks. Utilizing low-power wide-area networks (LPWAN) such as LoRaWAN ensures that remote sensor telemetry is delivered efficiently over long distances. To guarantee data integrity and prevent tampering, the system architecture must include edge computing capabilities on the local gateway device to encrypt payload data before transmission to the cloud server. This ensures that the Air Quality Index displayed on public dashboards remains a trusted source of information for community health decisions.

Extended discussion on IoT Environmental Sensor Calibration 9. The deployment of low-cost particulate matter sensors in urban environments requires rigorous calibration protocols to ensure data accuracy. Optical particle counters used for PM_{2.5} monitoring, while cost-effective, are sensitive to high humidity and aerosolized water droplets which can cause overestimation of pollution levels. Implementing a dynamic signal processing algorithm that corrects readings based on concurrent temperature and humidity data is crucial for minimizing false health alarms. Furthermore, sensor drift over time necessitates a robust device management model. By continuously comparing data against high-fidelity reference stations, the system can apply machine learning-based calibration curves over-the-air, ensuring uninterrupted and accurate environmental observation.

Detailed analysis of Smart City Communication Protocols and Data Security 9. In the event of a hazardous pollution anomaly, the speed, reliability, and security of the alert transmission to the public are paramount. Environmental data transmission must comply with strict smart city standards and data privacy frameworks. Utilizing low-power wide-area networks (LPWAN) such as LoRaWAN ensures that remote sensor telemetry is delivered efficiently over long distances. To guarantee data integrity and prevent tampering, the system architecture must include edge computing capabilities on the local gateway device to encrypt payload data before transmission to the cloud server. This ensures that the Air Quality Index displayed on public dashboards remains a trusted source of information for community health decisions.

Extended discussion on IoT Environmental Sensor Calibration 10. The deployment of low-cost particulate matter sensors in urban environments requires rigorous calibration protocols to ensure data accuracy. Optical particle counters used for PM2.5 monitoring, while cost-effective, are sensitive to high humidity and aerosolized water droplets which can cause overestimation of pollution levels. Implementing a dynamic signal processing algorithm that corrects readings based on concurrent temperature and humidity data is crucial for minimizing false health alarms. Furthermore, sensor drift over time necessitates a robust device management model. By continuously comparing data against high-fidelity reference stations, the system can apply machine learning-based calibration curves over-the-air, ensuring uninterrupted and accurate environmental observation.

Detailed analysis of Smart City Communication Protocols and Data Security 10. In the event of a hazardous pollution anomaly, the speed, reliability, and security of the alert transmission to the public are paramount. Environmental data transmission must comply with strict smart city standards and data privacy frameworks. Utilizing low-power wide-area networks (LPWAN) such as LoRaWAN ensures that remote sensor telemetry is delivered efficiently over long distances. To guarantee data integrity and prevent tampering, the system architecture must include edge computing capabilities on the local gateway device to encrypt payload data before transmission to the cloud server. This ensures that the Air Quality Index displayed on public dashboards remains a trusted source of information for community health decisions.

Extended discussion on IoT Environmental Sensor Calibration 11. The deployment of low-cost particulate matter sensors in urban environments requires rigorous calibration protocols to ensure data accuracy. Optical particle counters used for PM_{2.5} monitoring, while cost-effective, are sensitive to high humidity and aerosolized water droplets which can cause overestimation of pollution levels. Implementing a dynamic signal processing algorithm that corrects readings based on concurrent temperature and humidity data is crucial for minimizing false health alarms. Furthermore, sensor drift over time necessitates a robust device management model. By continuously comparing data against high-fidelity reference stations, the system can apply machine learning-based calibration curves over-the-air, ensuring uninterrupted and accurate environmental observation.

Detailed analysis of Smart City Communication Protocols and Data Security 11. In the event of a hazardous pollution anomaly, the speed, reliability, and security of the alert transmission to the public are paramount. Environmental data transmission must comply with strict smart city standards and data privacy frameworks. Utilizing low-power wide-area networks (LPWAN) such as LoRaWAN ensures that remote sensor telemetry is delivered efficiently over long distances. To guarantee data integrity and prevent tampering, the system architecture must include edge computing capabilities on the local gateway device to encrypt payload data before transmission to the cloud server. This ensures that the Air Quality Index displayed on public dashboards remains a trusted source of information for community health decisions.

Extended discussion on IoT Environmental Sensor Calibration 12. The deployment of low-cost particulate matter sensors in urban environments requires rigorous calibration protocols to ensure data accuracy. Optical particle counters used for PM_{2.5} monitoring, while cost-effective, are sensitive to high humidity and aerosolized water droplets which can cause overestimation of pollution levels. Implementing a dynamic signal processing algorithm that corrects readings based on concurrent temperature and humidity data is crucial for minimizing false health alarms. Furthermore, sensor drift over time necessitates a robust device management model. By continuously comparing data against high-fidelity reference stations, the system can apply machine learning-based calibration curves over-the-air, ensuring uninterrupted and accurate environmental observation.

Detailed analysis of Smart City Communication Protocols and Data Security 12. In the event of a hazardous pollution anomaly, the speed, reliability, and security of the alert transmission to the public are paramount. Environmental data transmission must comply with strict smart city standards and data privacy frameworks. Utilizing low-power wide-area networks (LPWAN) such as LoRaWAN ensures that remote sensor telemetry is delivered efficiently over long distances. To guarantee data integrity and prevent tampering, the system architecture must include edge computing capabilities on the local gateway device to encrypt payload data before transmission to the cloud server. This ensures that the Air Quality Index displayed on public dashboards remains a trusted source of information for community health decisions.

Extended discussion on IoT Environmental Sensor Calibration 13. The deployment of low-cost particulate matter sensors in urban environments requires rigorous calibration protocols to ensure data accuracy. Optical particle counters used for PM2.5 monitoring, while cost-effective, are sensitive to high humidity and aerosolized water droplets which can cause overestimation of pollution levels. Implementing a dynamic signal processing algorithm that corrects readings based on concurrent temperature and humidity data is crucial for minimizing false health alarms. Furthermore, sensor drift over time necessitates a robust device management model. By continuously comparing data against high-fidelity reference stations, the system can apply machine learning-based calibration curves over-the-air, ensuring uninterrupted and accurate environmental observation.

Detailed analysis of Smart City Communication Protocols and Data Security 13. In the event of a hazardous pollution anomaly, the speed, reliability, and security of the alert transmission to the public are paramount. Environmental data transmission must comply with strict smart city standards and data privacy frameworks. Utilizing low-power wide-area networks (LPWAN) such as LoRaWAN ensures that remote sensor telemetry is delivered efficiently over long distances. To guarantee data integrity and prevent tampering, the system architecture must include edge computing capabilities on the local gateway device to encrypt payload data before transmission to the cloud server. This ensures that the Air Quality Index displayed on public dashboards remains a trusted source of information for community health decisions.

Extended discussion on IoT Environmental Sensor Calibration 14. The deployment of low-cost particulate matter sensors in urban environments requires rigorous calibration protocols to ensure data accuracy. Optical particle counters used for PM_{2.5} monitoring, while cost-effective, are sensitive to high humidity and aerosolized water droplets which can cause overestimation of pollution levels. Implementing a dynamic signal processing algorithm that corrects readings based on concurrent temperature and humidity data is crucial for minimizing false health alarms. Furthermore, sensor drift over time necessitates a robust device management model. By continuously comparing data against high-fidelity reference stations, the system can apply machine learning-based calibration curves over-the-air, ensuring uninterrupted and accurate environmental observation.

Detailed analysis of Smart City Communication Protocols and Data Security 14. In the event of a hazardous pollution anomaly, the speed, reliability, and security of the alert transmission to the public are paramount. Environmental data transmission must comply with strict smart city standards and data privacy frameworks. Utilizing low-power wide-area networks (LPWAN) such as LoRaWAN ensures that remote sensor telemetry is delivered efficiently over long distances. To guarantee data integrity and prevent tampering, the system architecture must include edge computing capabilities on the local gateway device to encrypt payload data before transmission to the cloud server. This ensures that the Air Quality Index displayed on public dashboards remains a trusted source of information for community health decisions.

Extended discussion on IoT Environmental Sensor Calibration 15. The deployment of low-cost particulate matter sensors in urban environments requires rigorous calibration protocols to ensure data accuracy. Optical particle counters used for PM_{2.5} monitoring, while cost-effective, are sensitive to high humidity and aerosolized water droplets which can cause overestimation of pollution levels. Implementing a dynamic signal processing algorithm that corrects readings based on concurrent temperature and humidity data is crucial for minimizing false health alarms. Furthermore, sensor drift over time necessitates a robust device management model. By continuously comparing data against high-fidelity reference stations, the system can apply machine learning-based calibration curves over-the-air, ensuring uninterrupted and accurate environmental observation.

Detailed analysis of Smart City Communication Protocols and Data Security 15. In the event of a hazardous pollution anomaly, the speed, reliability, and security of the alert transmission to the public are paramount. Environmental data transmission must comply with strict smart city standards and data privacy frameworks. Utilizing low-power wide-area networks (LPWAN) such as LoRaWAN ensures that remote sensor telemetry is delivered efficiently over long distances. To guarantee data integrity and prevent tampering, the system architecture must include edge computing capabilities on the local gateway device to encrypt payload data before transmission to the cloud server. This ensures that the Air Quality Index displayed on public dashboards remains a trusted source of information for community health decisions.

